Affinity metrics

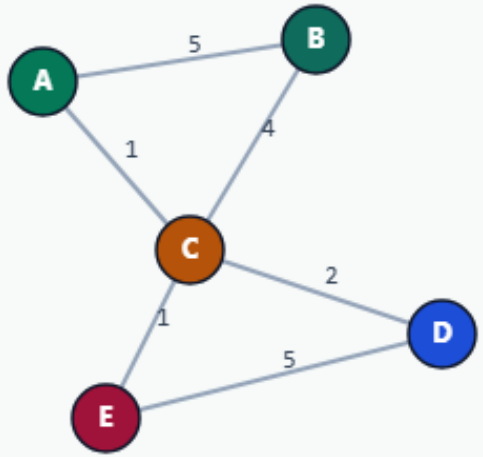
Before you merge cells or coarsen a netlist, you need a score that says these two should stay together

Start with a tiny weighted graph

AFFINITY METRICS

Start with a tiny weighted graph

Step 1 / 5 · raw-graph · module01-01-affinity-metrics



Explanation

Five cells A–E with edge weights. Heavy edges A–B and D–E both weigh 5; the bridge through C is weaker. Affinity will decide which pairs want to cluster first.

- Nodes = cells / clusters to group
- Edge weight = how strongly a pair should stay together
- Reference instance used across all clustering labs

```
Graph: 5 nodes, 6 edges
Heaviest edges: A–B=5, D–E=5
```

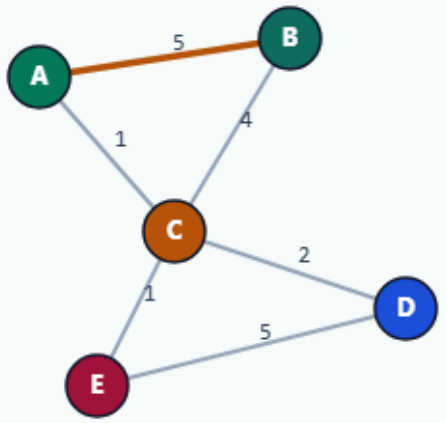
Start with a tiny weighted graph

Edge-weight affinity ranks pairs

AFFINITY METRICS

Edge-weight affinity ranks pairs

Step 2 / 5 · edge-rank · module01-01-affinity-metrics



Explanation

Pure edge-weight affinity just sorts existing edges. A–B and D–E tie at 5; tie-break puts A–B first. This ranking is the simplest merge priority.

- score(u,v) = w(u,v)
- Sort descending by score
- Top pair becomes the first merge candidate

```
1. A–B @ 5
2. D–E @ 5
3. B–C @ 4
4. C–D @ 2
5. A–C @ 1
6. C–E @ 1
```

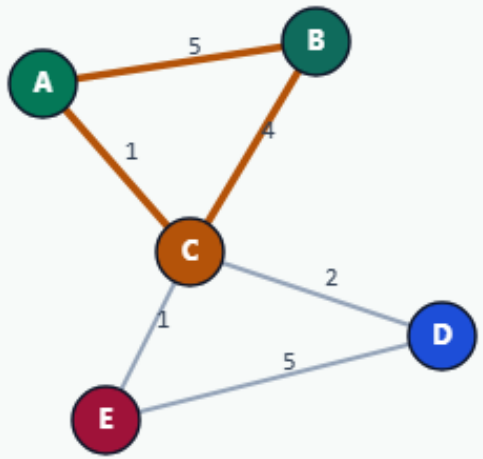
Edge-weight affinity ranks pairs

Shared neighbors boost A–B

AFFINITY METRICS

Shared neighbors boost A–B

Step 3 / 5 · shared-idea · module01-01-affinity-metrics



Explanation

A and B already share edge weight 5. They also share neighbor C. Shared-neighbor affinity adds min(w(A,C), w(B,C)) = min(1,4) = 1, so A–B rises to 6.

- Direct edge still counts
- Plus support through common neighbors
- Encourages dense triangles to cluster

```
w(A,B)=5
shared via C: min(1,4)=1
score(A,B)=6
```

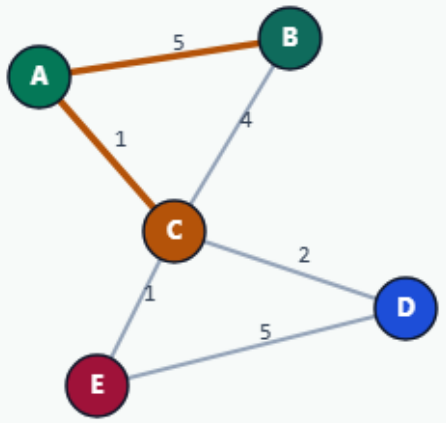
Shared neighbors boost A–B

Shared ranking reshuffles the list

AFFINITY METRICS

Shared ranking reshuffles the list

Step 4 / 5 · shared-rank · module01-01-affinity-metrics



Explanation

Under shared-neighbor scoring, A–B and D–E lead at 6. Weak edge A–C jumps to 5 because of B. Non-edges like B–D can appear from shared support alone.

- A–C: was 1 → now 5
- Invented pairs (e.g. B–D @ 2) show transitive pull
- Same idea feeds greedy merge priorities

```
1. A–B @ 6
2. D–E @ 6
3. A–C @ 5
4. B–C @ 5
5. C–D @ 3
6. C–E @ 3
```

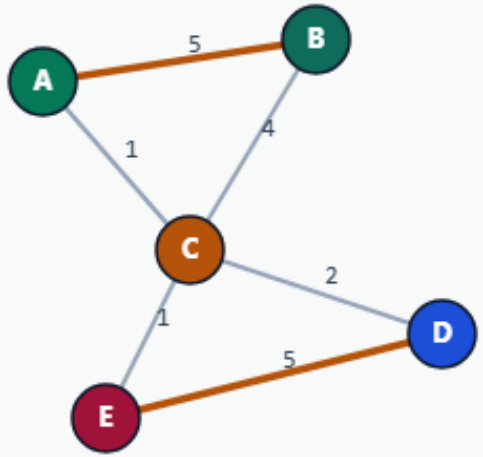
Shared ranking reshuffles the list

Why affinity matters in EDA

AFFINITY METRICS

Why affinity matters in EDA

Step 5 / 5 · takeaway · module01-01-affinity-metrics



Explanation

Affinity is the objective's local preference: which cells belong together before global constraints. Bad affinity → bad merges → harder refinement later.

- Edge weight = connectivity / nets shared
- Shared neighbors ≈ denser local groups
- Next labs consume this ranking to merge

```
Edge top: A–B @ 5
Shared top: A–B @ 6
Teaching graph stays fixed for goldens
```

Why affinity matters in EDA

Shared-neighbor scoring as pseudocode

- Here is the shared-neighbor score as code
- First, every existing edge adds its weight to its pair
- Then, for every pair of nodes, look at each neighbor they have in common
- If a pair ends up with a score above zero, keep it
- Finally, sort by score, highest first

Shared-neighbor scoring as pseudocode — try these

```
affinity_shared_neighbors(nodes, edges):
  score[u,v] = w(u,v) for every edge
  for every pair (u, v) of nodes:
    s = 0
    for x in neighbors(u), x != v:
      if x in neighbors(v):
        s += min(w(u,x), w(v,x))
    if s > 0 or (u,v) has an edge:
      score[u,v] += s
  sort by score (high first), then u, then v
```

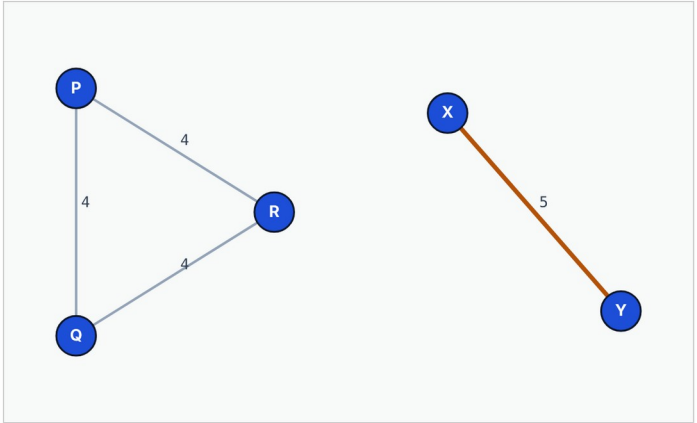
Ten pairs, not six

- Under shared-neighbor scoring the starter graph produces ten pairs, not six
- Four of them have no edge at all
- B and D both connect to C, so B–D scores the smaller spoke, two
- A–D, A–E and B–E each pick up one through C
- These pairs aren't mistakes: they say two cells are close even without a direct wire
- A pair is only left out when its score would be zero

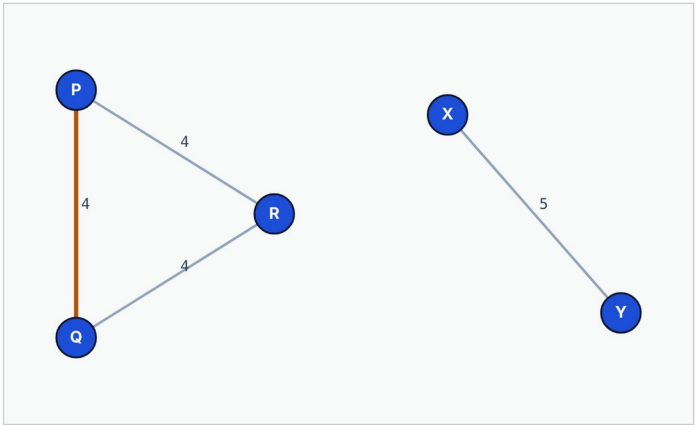
Same graph, different first merge

- Here's why the choice matters
- Take a tight triangle P, Q, R, with every edge at weight four
- Edge weight picks X–Y first, because five beats four
- Shared-neighbor scoring gives P–Q four plus four through R: eight
- X–Y has no common neighbor, so it stays at five
- Same graph, same merge engine, different first merge

Edge weight: first merge X–Y (5)



Shared neighbors: first merge P–Q (4+4 = 8)



Nets with many pins

- Real netlists aren't simple edges
- One net can connect many cells, like a reset or an enable line
- If you count every shared net at full weight
- Pin-count scoring fixes this
- Each net splits its weight across its pins: weight divided by pins minus one
- A two-pin net counts fully

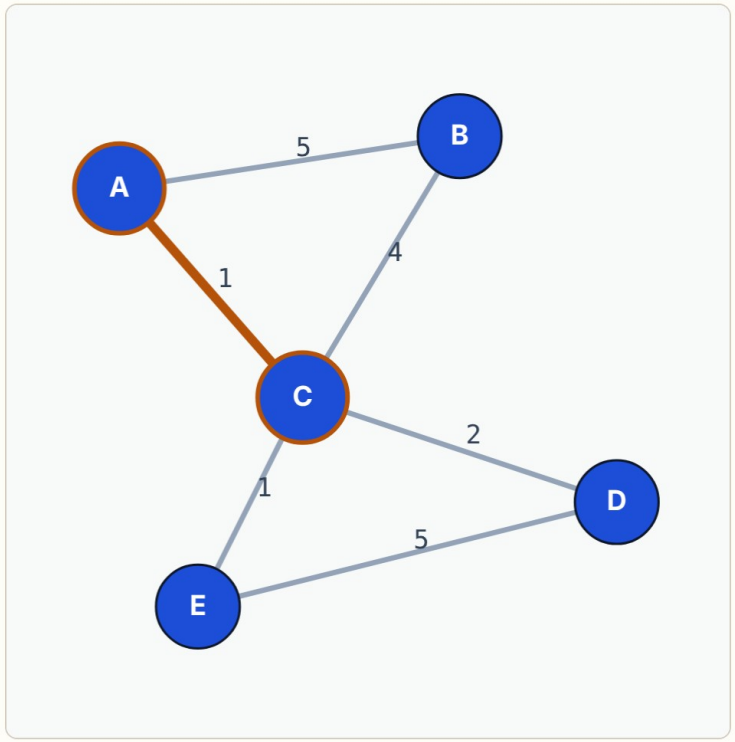
Nets with many pins — try these

```
tiny netlist: en {A,B,C,D,E}, rst {A,B,C,D,E}, n1 {E,F}

           raw net count     pin-count w/(|e|-1)
A-B        1 + 1 = 2         1/4 + 1/4 = 0.5
E-F        1                 1/1 = 1.0
first      A-B               E-F
```

Browser lab track

Graph



Reference / metrics

Pair A–C · score: Shared neighbors + edge

5 | Check my score

Add to my ranking | Show breakdown (study)

edge weight 1: − 1 | + 1

Correct: A–C = 5

```
graph: starter · score: Shared neighbors + edge

your scores:
  B–D: 2 ✓
  A–C: 5 ✓

your ranking:
  1. A–B
  2. D–E

weight edits (total change 0):
  (none)
```

Undo last ranking entry

Reveal full ranking (study)

Browser lab starter

Implement track

wsl — module01-01-affinity-metrics/examples/

```
# Track A — real Unix

$ python3 ../common/solvers.py examples/priority_graph.json --mode affinity
affinity edge-weight (desc):
  X-Y: 5.0
  P-Q: 4.0
  P-R: 4.0
  Q-R: 4.0
affinity shared-neighbors+edge (desc):
  P-Q: 8.0
  P-R: 8.0
  Q-R: 8.0
  X-Y: 5.0

$ python3 examples/affinity_check.py
checking reference
PASS  tiny graph, edge weight: 6 pairs in golden order
PASS  tiny graph, shared neighbors: 10 pairs in golden order
PASS  priority graph, edge weight: first merge X-Y @ 5
PASS  priority graph, shared neighbors: first merge P-Q @ 8
PASS  netlist, pin-count: first merge E-F @ 1, A-B only 0.5
all goldens match
```

Real shell session

Implement track — try these

```
# point Python at the shared helpers
export PYTHONPATH=../common

# both graph rankings, starter graph then priority graph
python3 ../common/solvers.py examples/tiny_graph.json --mode
affinity
python3 ../common/solvers.py examples/priority_graph.json --mode
affinity

# check every golden against your own functions
python3 examples/affinity_check.py --impl my_affinity.py
```

Pitfalls to watch

- Watch for four things
- Counting an undirected edge twice, once from each end
- Adding the larger spoke instead of the smaller one, which makes everything look close
- Forgetting the tie-break, so A–B and D–E swap places from run to run
- And using raw net counts on a netlist

Your turn

- Match all three golden checks, then explain two things in your own words
- Take the short quiz